Examen Práctico de Servicios

Billetera Digital

Autor: Rómulo Palacios Rivas

Rol: QA Tester

Estudiante de Ingeniería de Ingeniería en Tecnologías de la Información

Fecha: 8 de marzo de 2026

Contenido

PASO 1 3
Configuración Inicial y Creación de Identidades 3
1. Configuración del Entorno (Environment): 3
2. Creación y Gestión de Usuario 1 (EMISOR): 3
3. Creación y Gestión de Usuario 2 (RECEPTOR): 4
PASO 2 6
Automatización de la Autenticación 6
1. Login de Usuario 1: 6
2. Login de Usuario 2: 7
PASO 3: 9
Aprovisionamiento de Fondos con Validación 9
1. Saldo: 9
2. Recarga: 10
PASO 4: 12
Flujo de Transferencia de Fondos 12
1. Transferencia: 12
1.1. Escenario Negativo: Saldo Insuficiente 12
1.2. Escenario Positivo: Flujo Exitoso 13
PASO 5: 17
Validación de Recepción por el Receptor 17
1. Consulta de Historial: 17
PASO 6: 19
Conclusiones 19
Github repo: 19

PASO 1

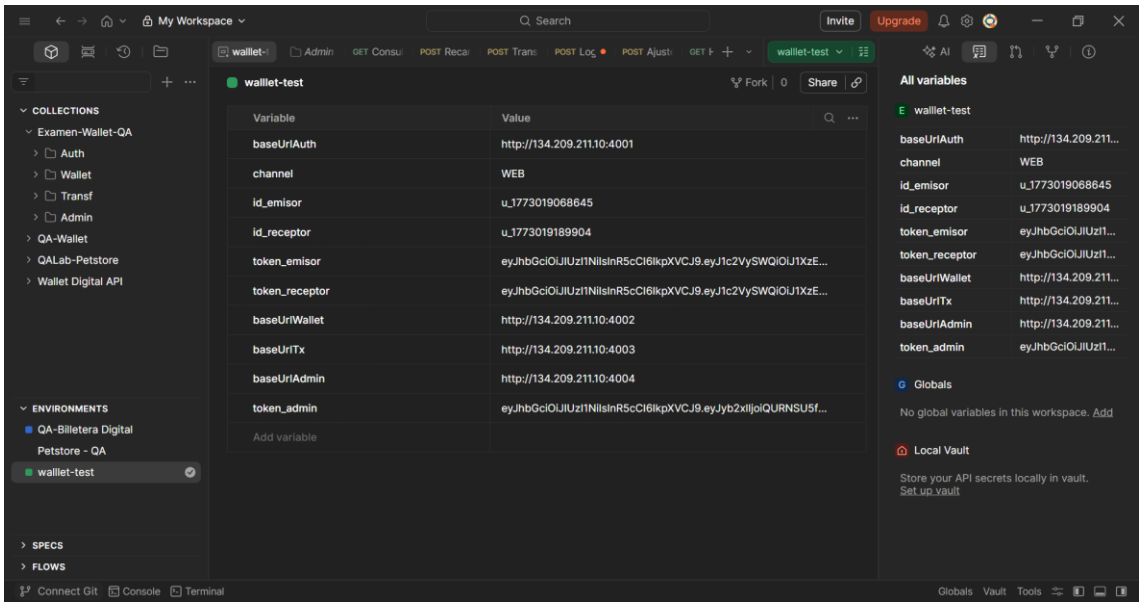
Configuración Inicial y Creación de Identidades

1. Configuración del Entorno (Environment):

Para garantizar la escalabilidad y el dinamismo de las pruebas, se configuró un entorno en Postman denominado *wallet-test*.

Se identificó que la arquitectura opera bajo múltiples microservicios, por lo que se creó las siguientes variables clave:

- baseUrlAuth: Puerto 4001 (Servicios Autenticación)
- baseUrlWallet: Puerto 4002 (Servicios de Billetera)
- baseUrlTx: Puerto 4003 (Transferencia)
- baseUrlAdmin: Puerto 4004 (Admin)
- channel: WEB (Header obligatorio de seguridad)
- Variables dinámicas para IDs y Tokens (id_emisor, id_receptor, token_emisor, etc.).

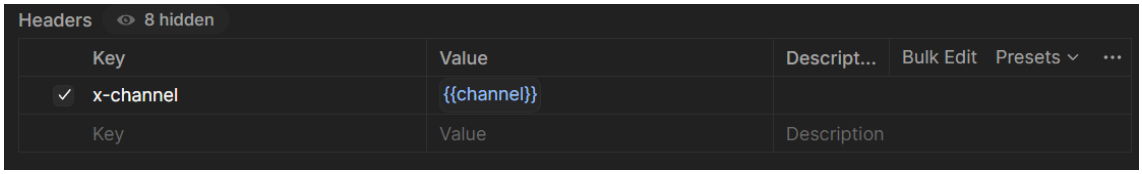


2. Creación y Gestión de Usuario 1 (EMISOR):

El objetivo de este request fue registrar al usuario que enviará los fondos.

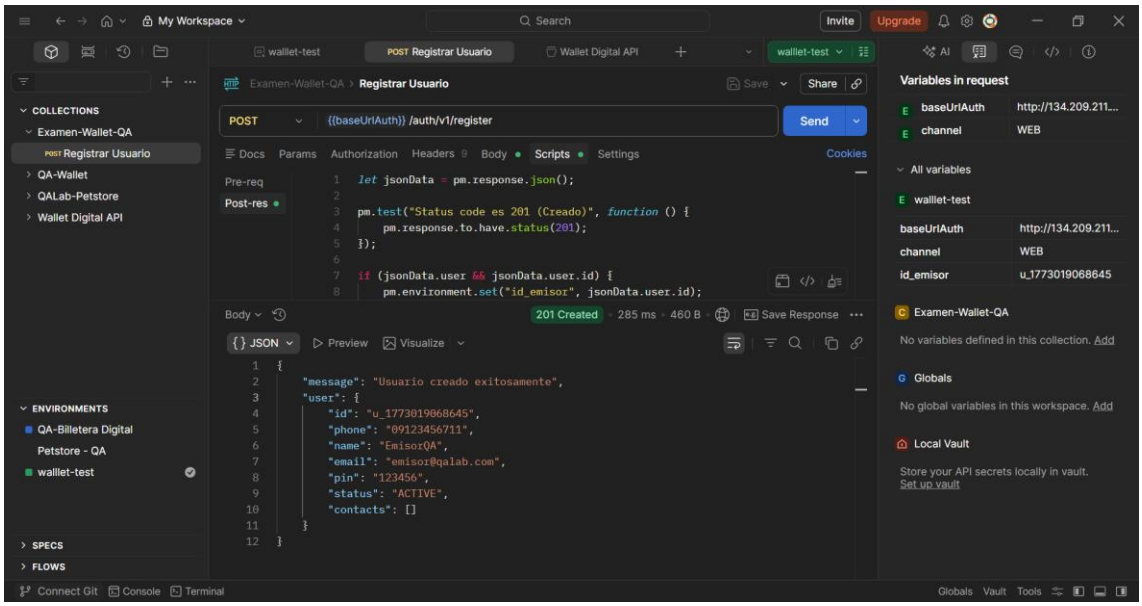
Endpoint: ***POST*** *{{baseUrlAuth}}/auth/v1/register*

2.1. Headers: Se configuró el key x-channel con el valor dinámico {{channel}} (previamente definido como WEB en el entorno).



2.2.Validación y Automatización (Tests): Se implementó un script para realizar una aserción del código de estado esperado y extraer el "id_emisor" generado hacia las variables de entorno, evitando la intervención manual.

2.3.Body: Se estructuró un objeto JSON con datos únicos para evitar conflictos (409 Conflict).



3. Creación y Gestión de Usuario 2 (RECEPTOR):

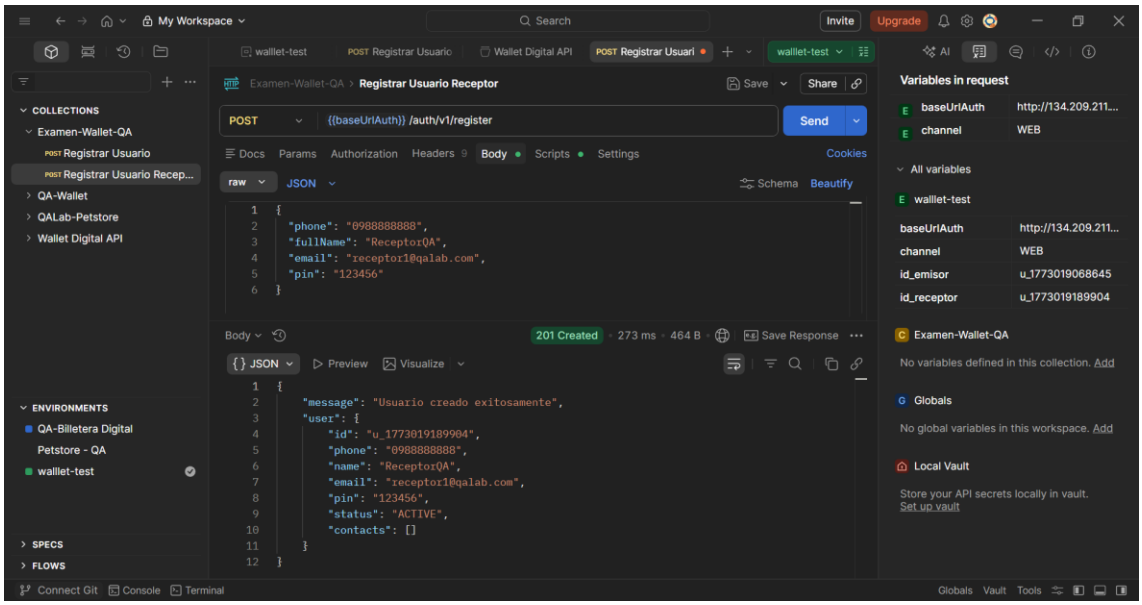
Se duplicó la estructura del request anterior para generar al usuario destinatario de la transferencia, modificando los datos de entrada y las variables de salida.

Endpoint: ***POST*** *{{baseUrlAuth}}/auth/v1/register*

3.1.Headers: Se configuró el key x-channel con el valor dinámico {{channel}} (previamente definido como WEB en el entorno).

3.2.Validación y Automatización (Tests): Se implementó un script para realizar una aserción del código de estado esperado y extraer el "id_receptor" generado hacia las variables de entorno, evitando la intervención manual.

3.3.Body: Se estructuró un objeto JSON con datos únicos para evitar conflictos (409 Conflict



PASO 2

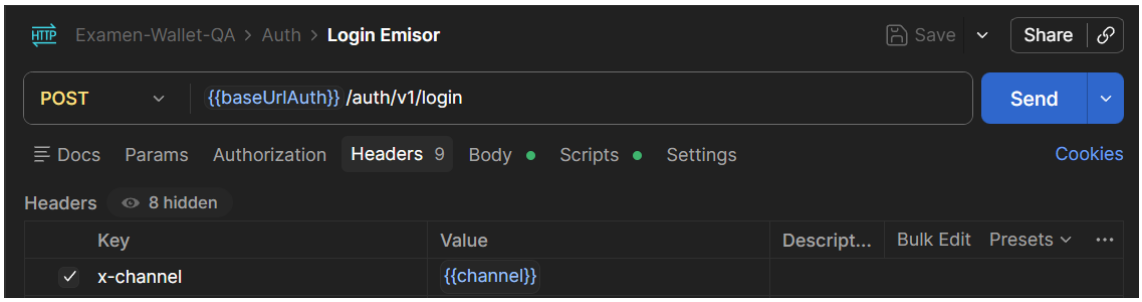
Automatización de la Autenticación

1. Login de Usuario 1:

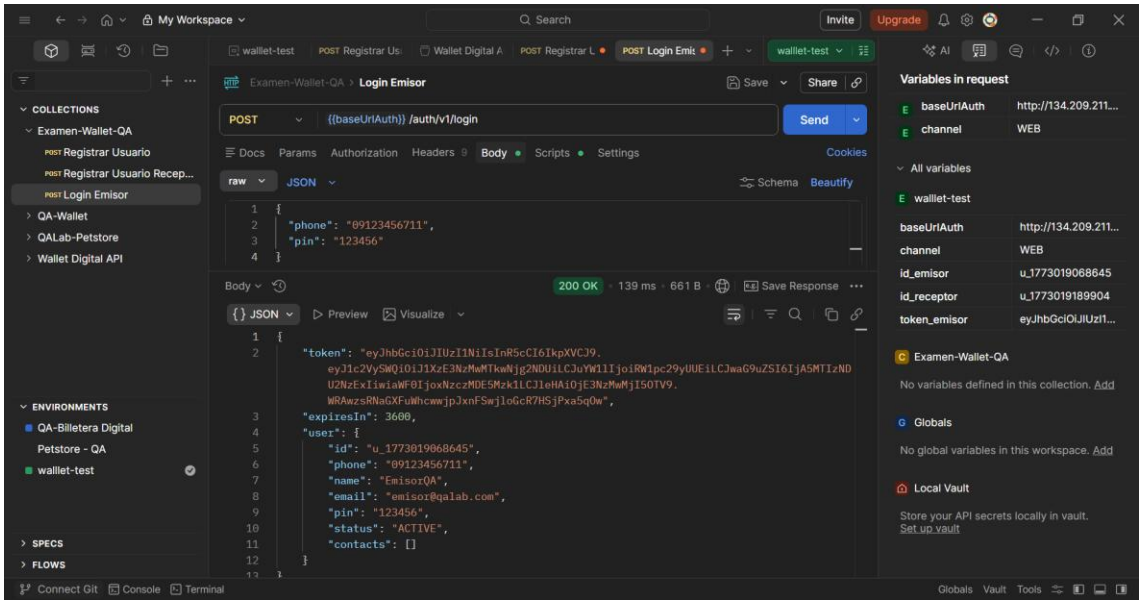
Se ejecutó la petición de inicio de sesión utilizando las credenciales generadas en el paso anterior.

*Endpoint: **POST** {{baseUrlAuth}}/auth/v1/login*

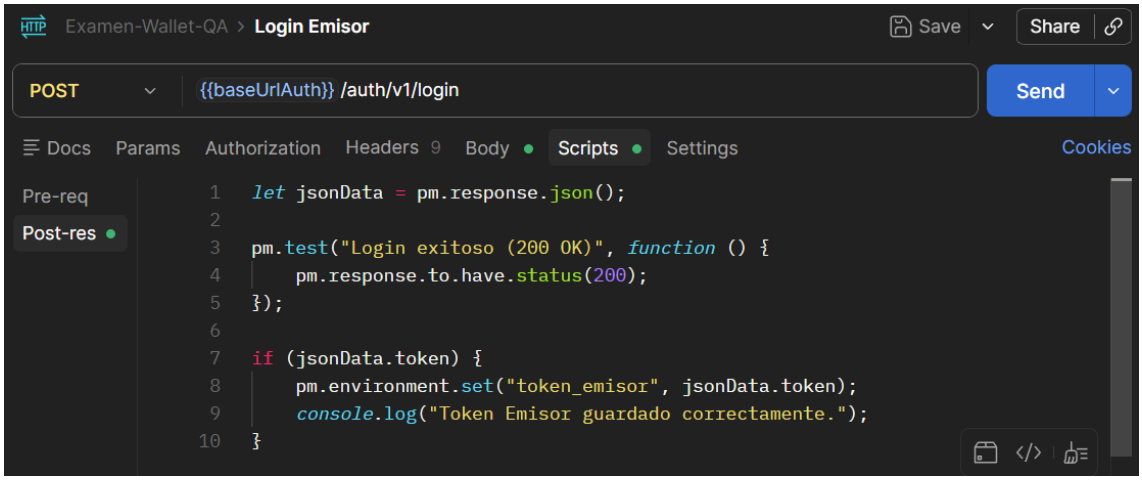
1.1.Headers: Se incluyó el header obligatorio de trazabilidad x-channel: {{channel}}.



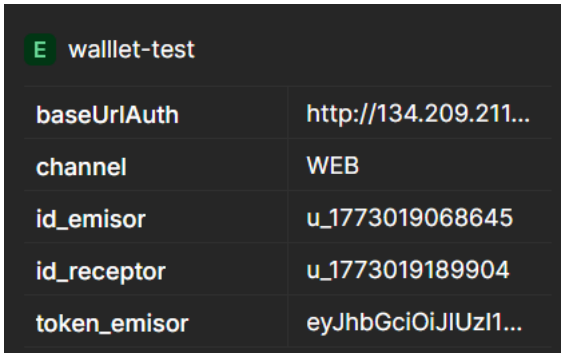
1.2.Body: Se enviaron las credenciales del emisor en formato JSON.



1.3. Validación y Automatización (Tests): Se desarrolló un script de pruebas para validar la respuesta HTTP 200 OK y extraer el Token JWT en “token_emisor” de forma dinámica.



1.4. Esto asegura que el token se asigne a la variable de entorno correspondiente sin intervención manual, facilitando la ejecución continua de la colección.

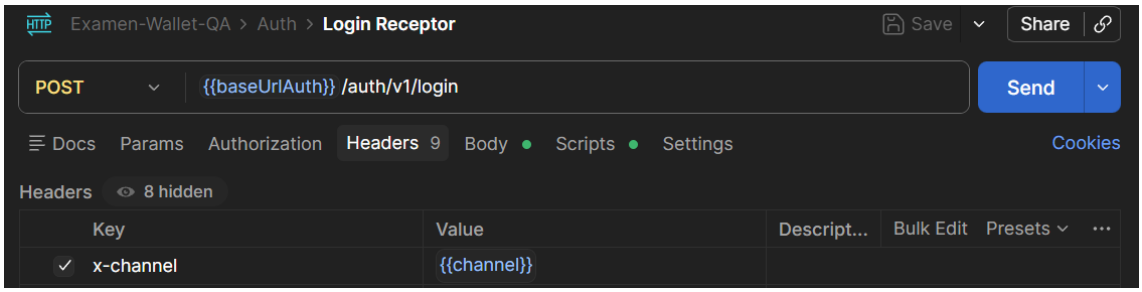


2. Login de Usuario 2:

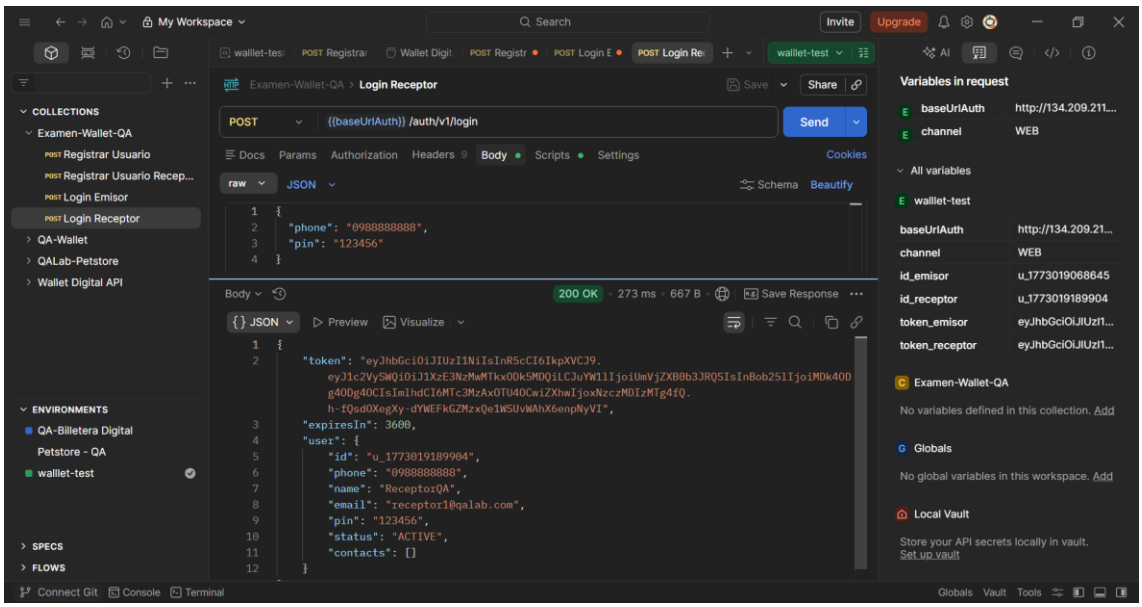
Para poder verificar la recepción de los fondos al final del flujo, fue necesario autenticar también al usuario destinatario y almacenar su token en una variable independiente.

Endpoint: ***POST*** *{{baseUrlAuth}}/auth/v1/login*

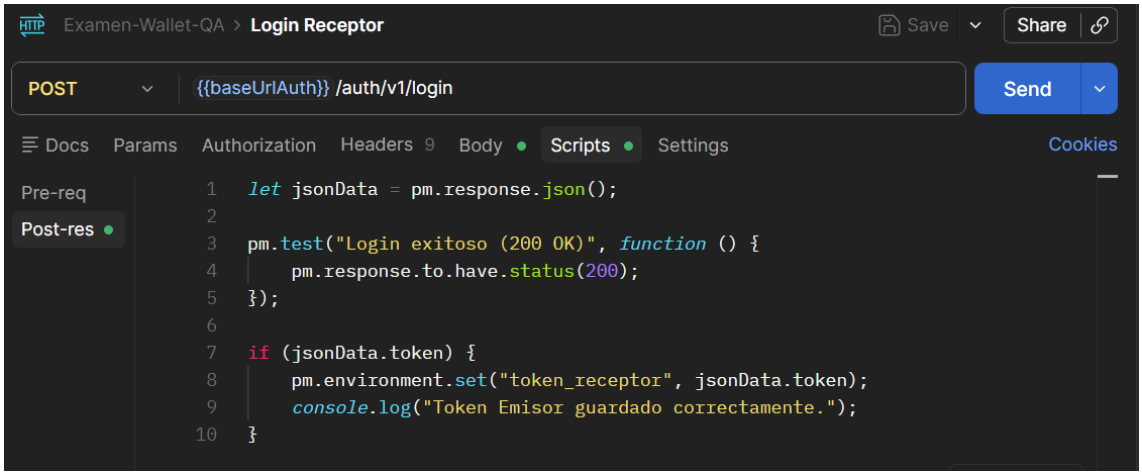
2.1.Headers: x-channel: {{channel}}



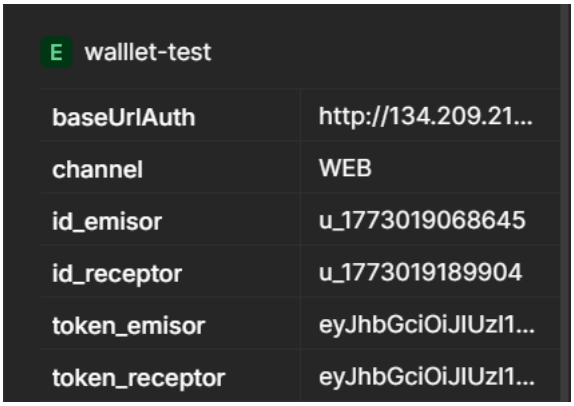
2.2.Body:



2.3.Validación y Automatización (Tests): El script se adaptó para almacenar el token en la variable token_receptor, manteniendo la encapsulación y previniendo la sobreescritura de la sesión del emisor.



2.4. Esto asegura que el token se asigne a la variable de entorno correspondiente sin intervención manual, facilitando la ejecución continua de la colección.



PASO 3:

Aprovisionamiento de Fondos con Validación

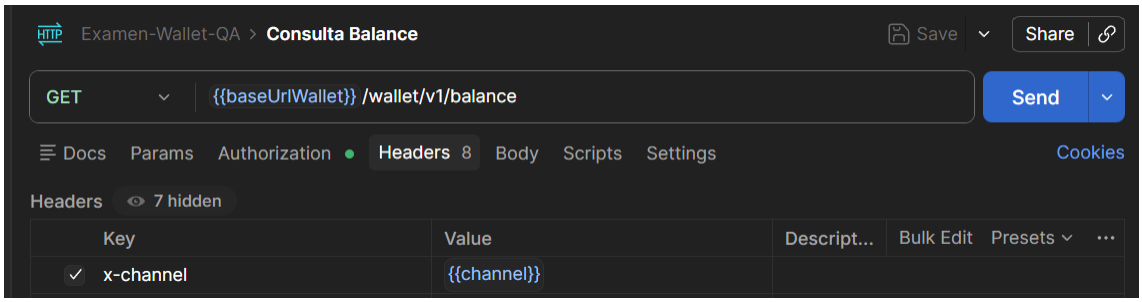
Una vez obtenidas las credenciales de acceso (Tokens JWT), se procedió a interactuar con el Wallet Service (puerto 4002) para fondear la cuenta del Emisor. El objetivo de este paso fue simular una recarga de dinero externa y aplicar las aserciones obligatorias para validar la operación.

1. Saldo:

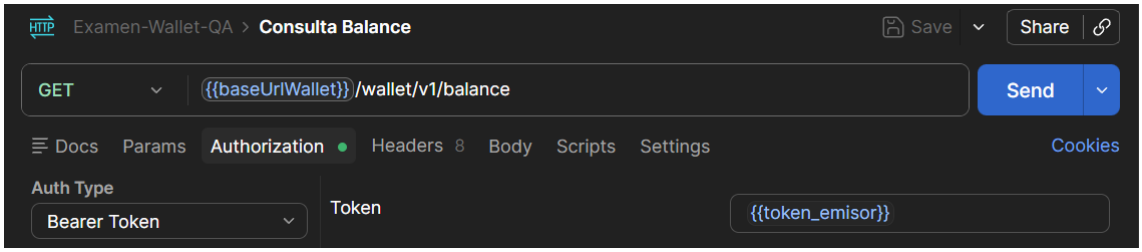
Antes de realizar una recarga, se verificó el estado inicial de la billetera del usuario.

*Endpoint: **GET** {{baseUrlWallet}}/wallet/v1/balance.*

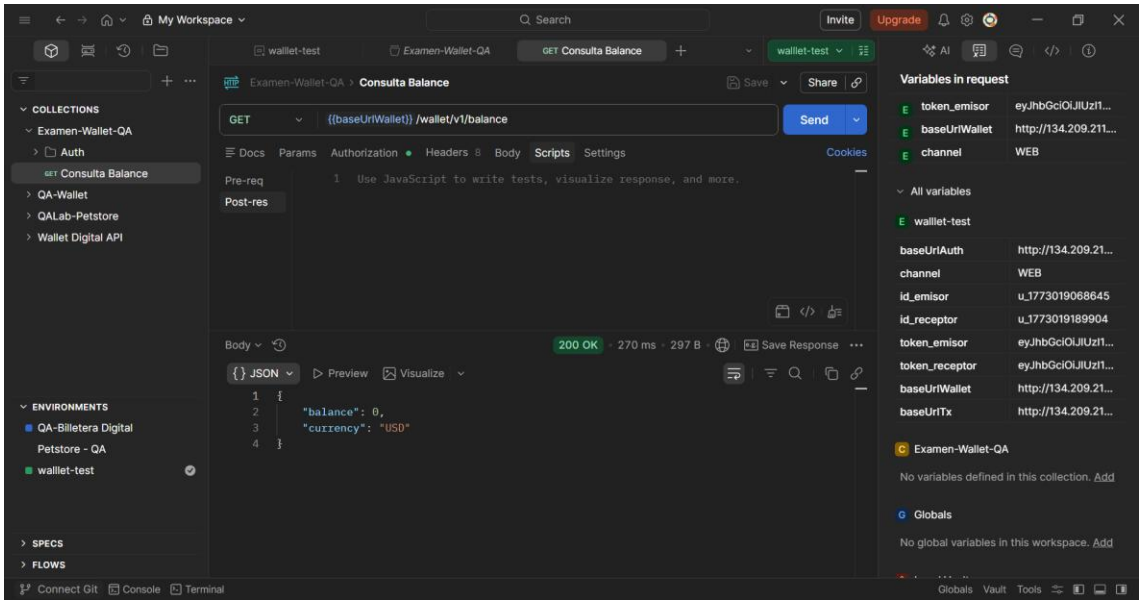
1.1.Headers: Se incluyó x-channel: {{channel}}.



1.2.Authorization: Se configuró el tipo Bearer Token utilizando la variable dinámica {{token_emisor}} para autorizar la petición.



El sistema retornó el saldo actual del usuario validando que el token JWT funcionaba correctamente en los microservicios protegidos.

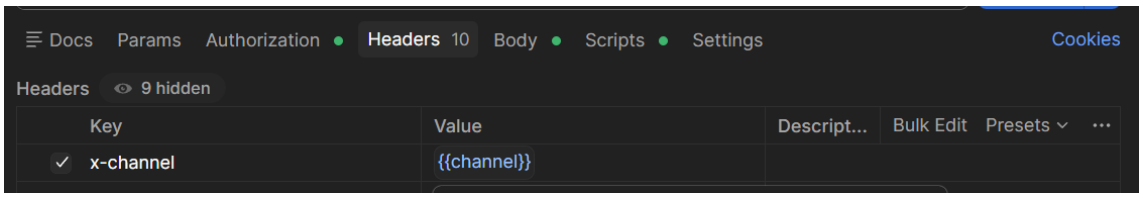


2. Recarga:

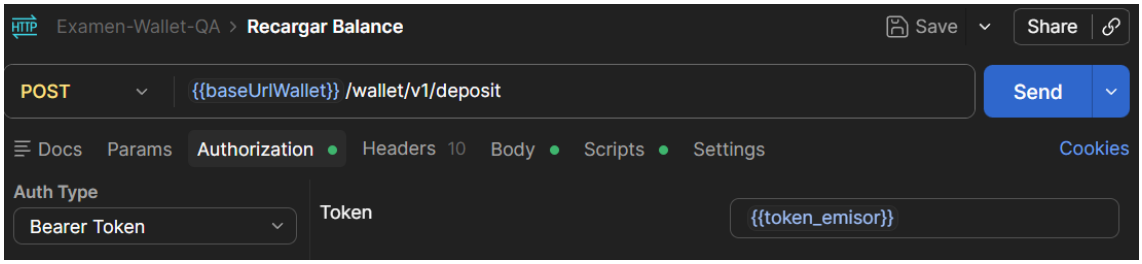
Se procedió a inyectar fondos a la cuenta del Emisor para preparar el escenario de la transferencia.

Endpoint: ***POST*** *{{baseUrlWallet}}/wallet/v1/deposit.*

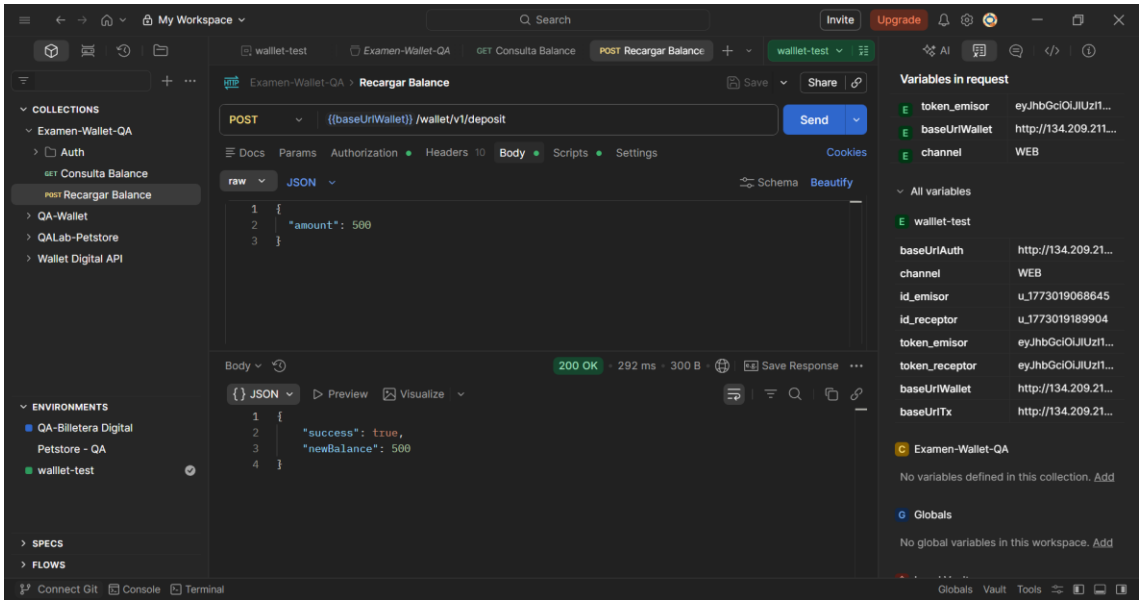
2.1.Headers: x-channel: {{channel}}.



2.2.Authorization: Tipo Bearer Token utilizando {{token_emisor}}.

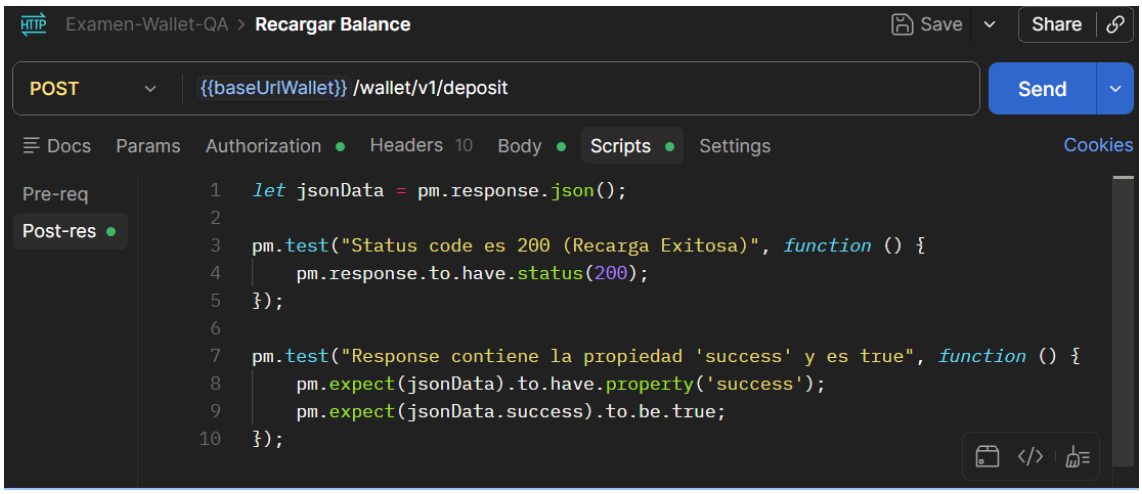


2.3.Body: Se estructuró un objeto JSON con el monto exacto requerido por el examen.



2.4.Validación y Automatización (Tests): Se añadieron Test Scripts para confirmar dos criterios de aceptación:

- Verificar que el código de estado HTTP fuera el esperado para una operación exitosa.
- Confirmar que la respuesta contuviera la propiedad booleana *success*.



PASO 4:

Flujo de Transferencia de Fondos

1. Transferencia:

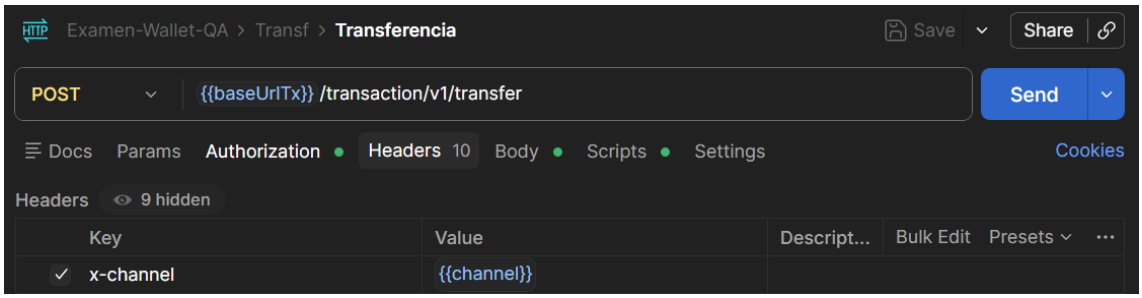
El objetivo principal de esta fase fue evaluar la lógica de negocio alojado en el Transaction Service (puerto 4003). Se solicitó transferir un monto de 950 desde el Emisor hacia el Receptor, utilizando variables de entorno dinámicas para la autorización y la identificación del destinatario.

1.1.Escenario Negativo: Saldo Insuficiente

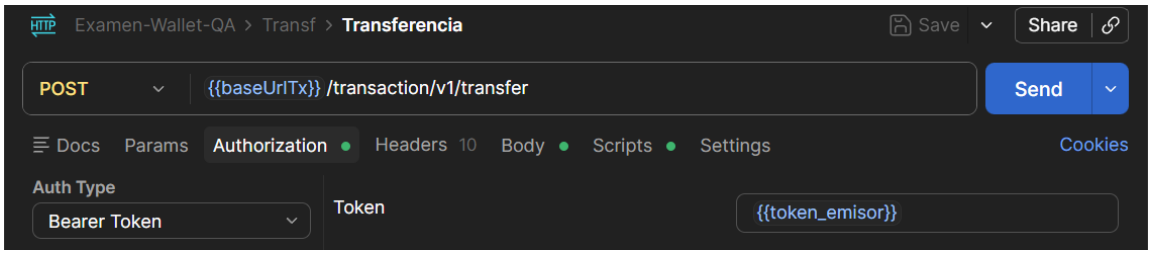
Al seguir estrictamente las instrucciones del examen, se contaba con un saldo de 500 pero se intentó procesar una salida de 950. Esto permitió evaluar los mecanismos de control del sistema.

*Endpoint: **POST** {{baseUrlTx}}/transaction/v1/transfer.*

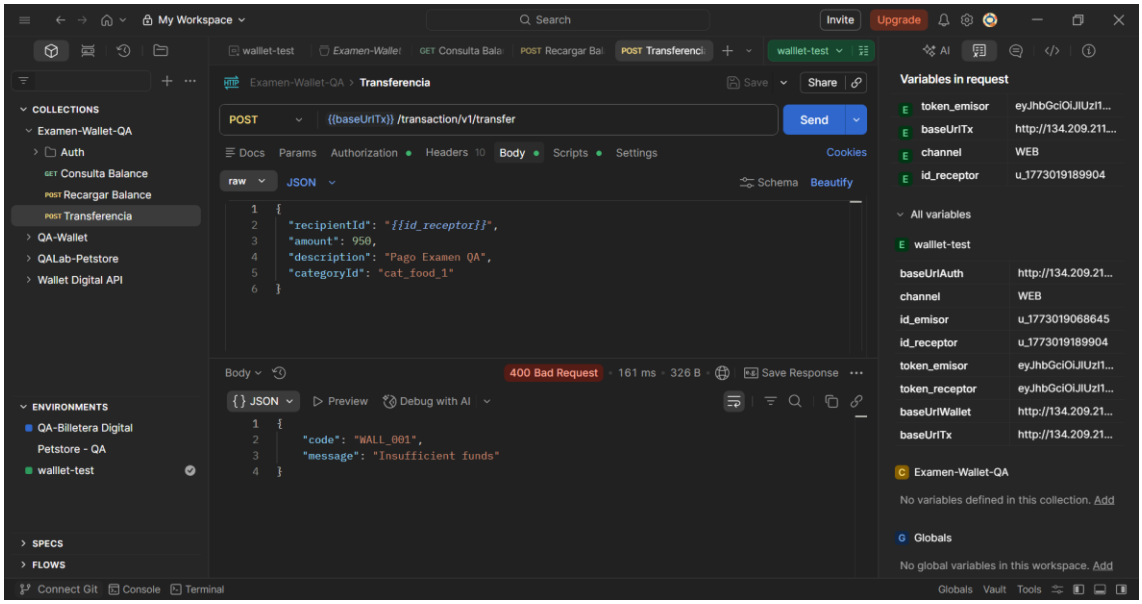
1.1.1. **Headers:** x-channel: {{channel}}.



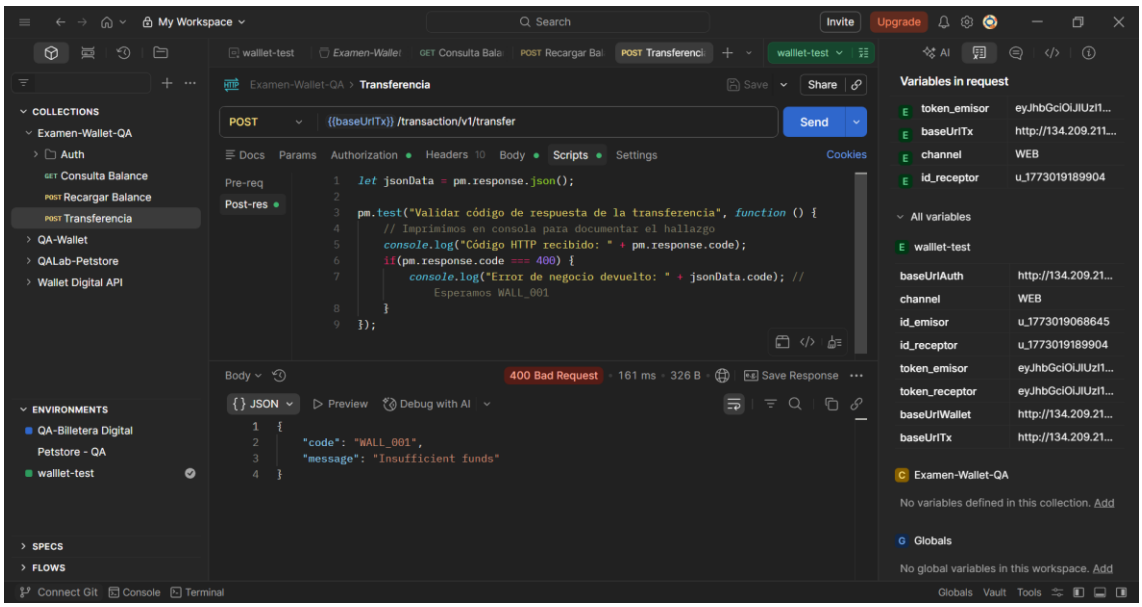
1.1.2. **Authorization:** Tipo Bearer Token utilizando {{token_emisor}} para identificar al originador de la transacción.



1.1.3. **Body:** Se inyectó dinámicamente la variable {{id_receptor}} como destinatario.



1.1.4. El sistema interceptó correctamente la operación, retornando un código HTTP 400 Bad Request. El payload de error indicó el código interno WALL_001 con el mensaje "Insufficient funds".

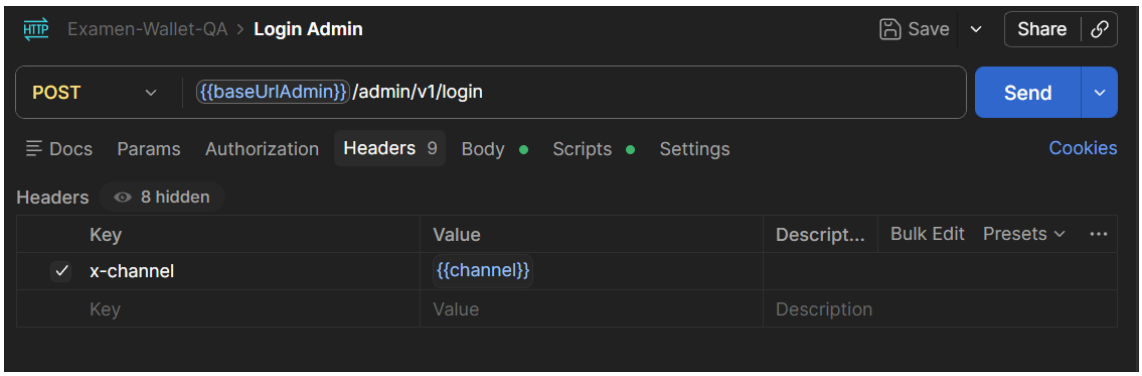


1.2.Escenario Positivo: Flujo Exitoso

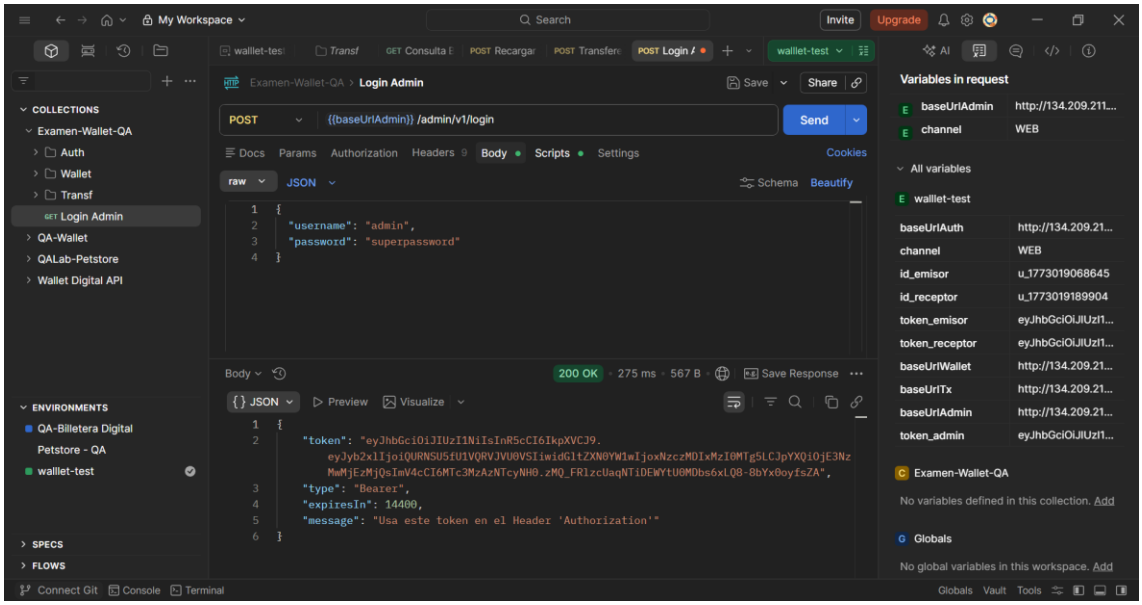
Para cumplir con el requerimiento estricto de transferir exactamente 950 y poder validar su recepción en el Paso 5, fue necesario subsanar el error WALL_001 inyectando fondos adicionales a la cuenta del Emisor. Como estrategia de pruebas, se consumió el Admin Service (puerto 4004) para realizar una acreditación manual.

Endpoint: ***POST*** *{{baseUrlAdmin}}/admin/v1/login.*

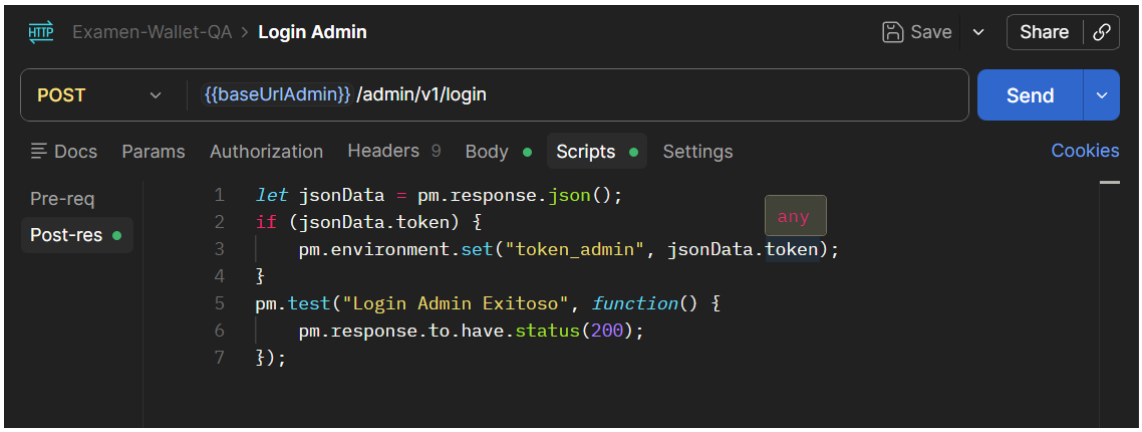
1.2.1. **Headers**: x-channel: {{channel}}.



1.2.2. **Body:** Se utilizaron las credenciales maestras especificadas en el contrato (username: "admin", password: "superpassword").



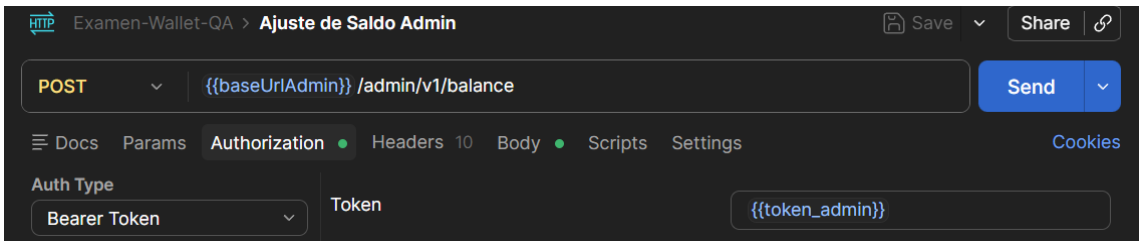
1.2.3. **Validación**: Se validó el código HTTP 200 OK y se almacenó el token de larga duración en la variable de entorno {{token_admin}} mediante un Test Script.



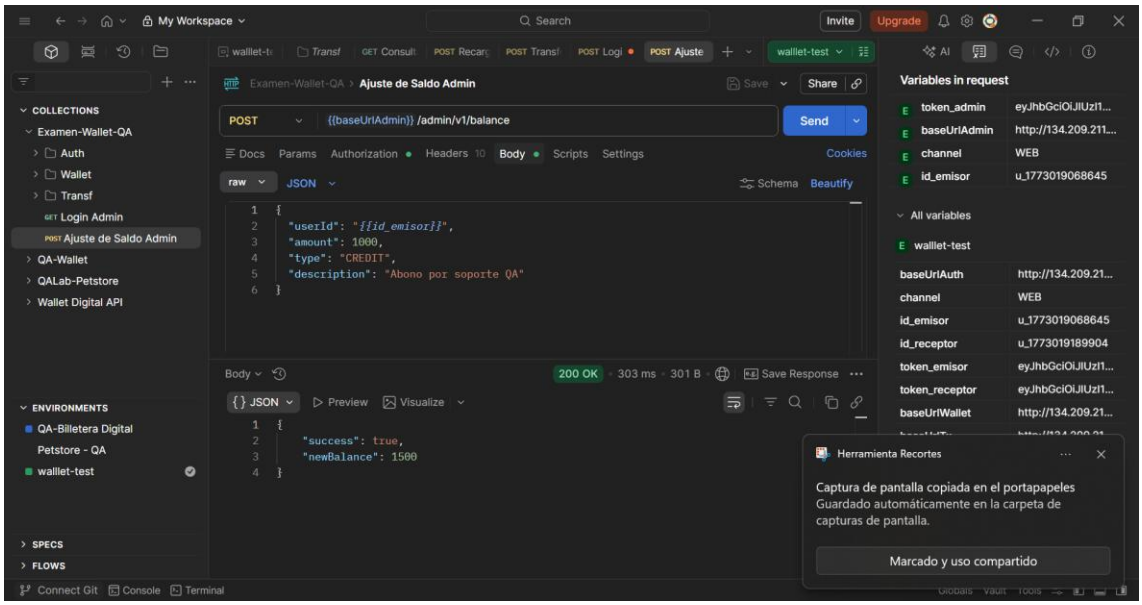
Se utilizó el endpoint de auditoría para forzar un crédito en la billetera del Emisor.

Endpoint: ***POST*** *{{baseUrlAdmin}}/admin/v1/balance.*

1.2.4. **Autenticación**: Bearer Token utilizando la variable {{token_admin}}.



1.2.5. Body:

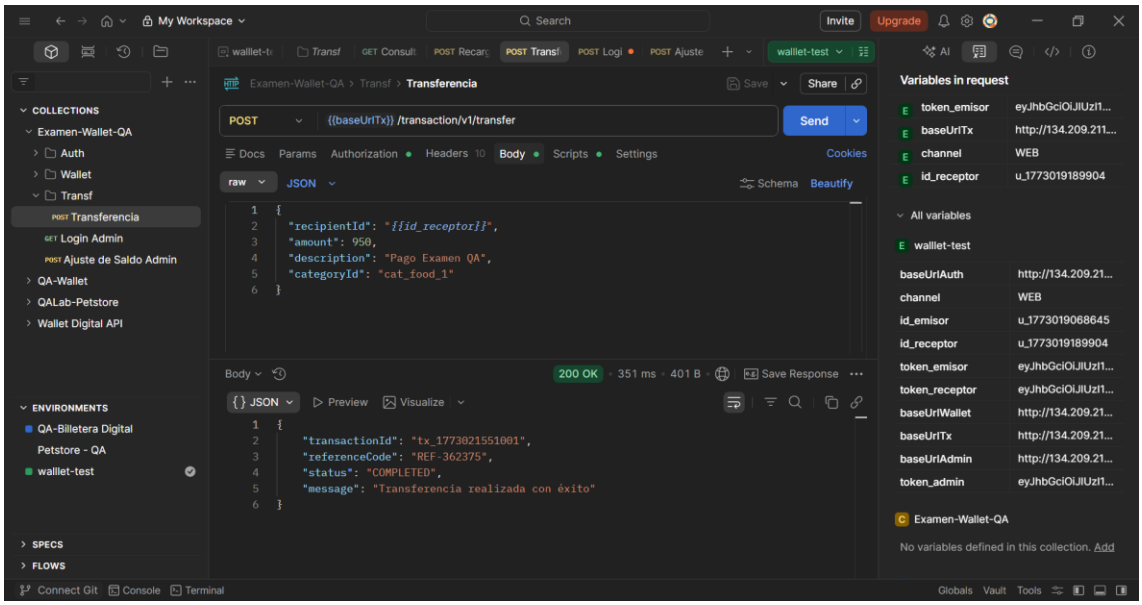


Con la cuenta correctamente fondeada, se retomó el flujo original del Paso 4.

Se volvió a enviar la petición

Endpoint: ***POST*** *{{baseUrlTx}}/transaction/v1/transfer*

El Transaction Service procesó la petición exitosamente sin activar la restricción de saldo, retornando un 200 OK.



Validación en Consola: El cuerpo de la respuesta incluyó un transactionId único y el estado COMPLETED, confirmando que el dinero salió de la cuenta del Emisor.

PASO 5:

Validación de Recepción por el Receptor

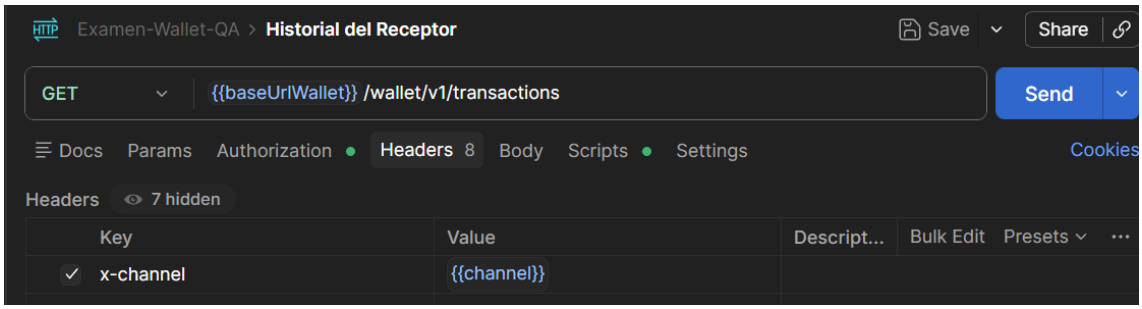
Tras completar la transferencia con éxito desde la cuenta del Emisor, la prueba exigió cambiar el contexto de la sesión para actuar como el Usuario 2 (Receptor). El objetivo fue confirmar la consistencia de los datos entre microservicios, asegurando que el Wallet Service reflejara el movimiento procesado por el Transaction Service.

1. Consulta de Historial:

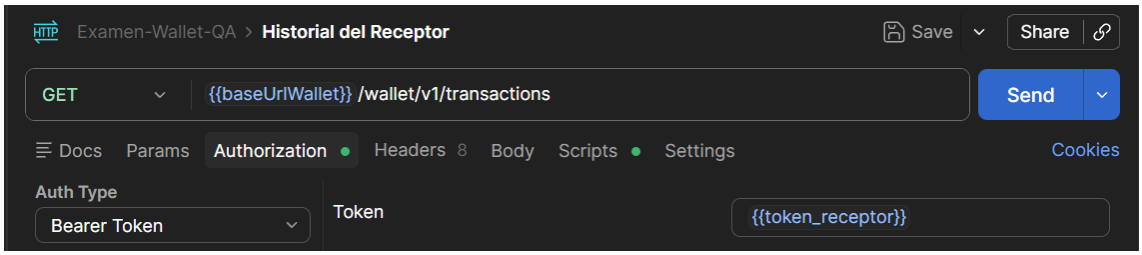
Se consultó el registro histórico del destinatario para verificar el ingreso de los fondos.

*Endpoint: **GET** {{baseUrlWallet}}/wallet/v1/transactions.*

1.1.Headers: Se mantuvo el estándar de seguridad con x-channel: {{channel}}.

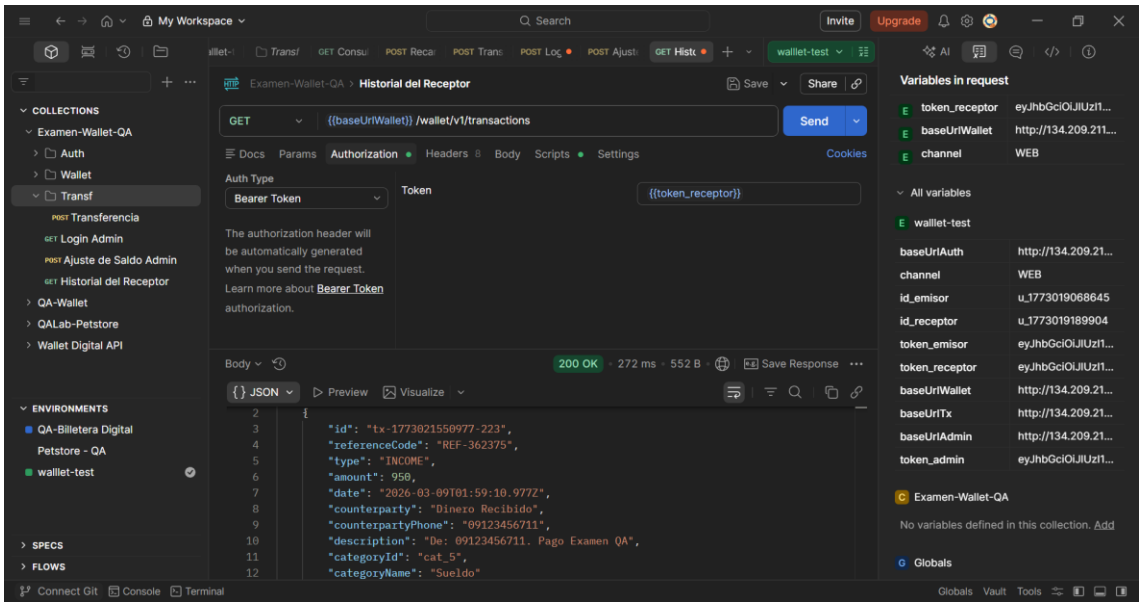


1.2.Authorization: Se configuró el tipo Bearer Token, pero esta vez utilizando estrictamente la variable {{token_receptor}} generada en el Paso 2. Esto garantizó que la consulta se realizara bajo la identidad y los permisos del destinatario.



1.3.Validación y Automatización (Tests): El requerimiento de la prueba especificaba confirmar que la transferencia fue recibida verificando que el listado de transacciones no estuviera vacío. Se programaron las siguientes aserciones:

El sistema retornó un código 200 OK. El cuerpo de la respuesta **Body** mostró un arreglo JSON conteniendo la transacción con el monto exacto de 950, clasificada bajo el tipo INCOME y con la descripción de la prueba.



PASO 6:

Conclusiones

Al empezar a realizar el examen, las instrucciones indicaban que la API funcionaba bajo la url proporcionada al inicio, con base en el puerto 5173. Sin embargo, al adentrarme más en el examen y realizar pruebas exploratorias, me di cuenta de que el sistema realmente está construido bajo una arquitectura de microservicio tal y como vimos en clase, donde cada módulo responde a un puerto distinto. Para darle solución de forma eficiente y no "hardcodear" las rutas, me di la tarea de configurar las variables de entorno independientes para cada servicio.

El escenario mas interesante para mi durante el examen, fue el seguir el flujo donde el sistema bloqueaba las transacciones sin fondos, el fujo pedía transferir $950, pero en el paso anterior solo se habían depositado 500. Al intentar hacer el envió, la API hizo exactamente lo que se esperaba, bloquear la transacción debido al saldo insuficiente de la cuenta, dando como resultado un bad request. Buscando una solución alternativa, se hizo uso de una alternativa para no romper el flujo original de recargar, la estrategia se basaba en autenticarme como admin usando su servicio de administrador para inyectarle saldo a la cuenta del emisor.

Durante todo el proceso, noté una API muy estricta, de carácter empresarial, enfrentándonos a un sistema de seguridad de la vida real. Si se intentaba hacer cualquier petición sin incluir el header obligatorio x-channel, el sistema inmeditamente nos enviaba un error GEN_001. Me parece una excelente práctica para proteger los endpoints y tener control de desde dónde se están realizando las peticiones.

Github repo:

https://github.com/romulopalacios/qa- examen-prueba-de-servicios.git